

# DARS: A Dynamic Adaptive Rating System with Graph-Guided Assessment and Remediation for Personalized Coding Education at Scale

Khethavath Sunil Naik

Computer Science Department  
Integrated BTech Research Initiative

**Abstract**—Online coding platforms have revolutionized technical education, yet accurate difficulty characterization remains elusive. This paper presents DARS, a comprehensive framework for dynamic problem difficulty prediction combining classical rating mechanisms, graph-guided feature engineering, and modern knowledge tracing methods. We evaluate DARS on the LeetCode dataset (1,825 problems, 20M+ users) systematically comparing six knowledge representation approaches: Fixed-Elo, Glicko-2, Bayesian Knowledge Tracing (BKT), Deep Knowledge Tracing (DKT), Self-Attentive Knowledge Tracing (SAKT), and Graph Neural Knowledge Tracing (GNKT). The framework incorporates four graph construction strategies—skill-only, temporal transition, expert prerequisite, and hybrid—evaluated across multiple fairness dimensions including learner group stratification, skill category analysis, and cold-start scenarios. DARS achieves 76.71% accuracy, 0.7485 AUC, and 0.0375 ECE on binary difficulty classification (Easy vs. Medium/Hard), substantially outperforming modern deep learning baselines (SAKT: 73.42%, DKT: 71.23%, GNKT: 72.05%). Key findings include: (1) interpretable feature engineering rivals black-box approaches while maintaining transparency, (2) expert-guided graph construction surpasses learned alternatives by 4.4 percentage points, (3) critical cold-start vulnerability affecting 22% of problems (low-acceptance items), (4) fairness gaps requiring mitigation (66.7% TPR disparity), and (5) feature frequency emerges as the single most critical predictor (9% performance loss upon removal). We further propose evaluation methodology for live adaptive assessment environments that measure learning outcomes beyond accuracy. This work bridges classical educational assessment with modern data science, demonstrating that principled feature engineering combined with graph-guided reasoning provides both interpretability and competitive performance for scaled educational systems.

## I. INTRODUCTION

### A. Motivation and Problem Statement

The proliferation of online learning platforms has created unprecedented opportunities for understanding how learners acquire technical skills. Platforms like LeetCode, HackerRank, and CodeSignal serve millions of learners daily, generating vast streams of interaction data. Yet despite this abundance, a fundamental challenge remains: accurately characterizing problem difficulty to enable personalized learning pathways.

Current approaches suffer critical limitations:

- 1) **Static Labels:** Hard-coded difficulty labels (Easy/Medium/Hard) are coarse-grained and inconsistent across platforms
- 2) **Learner Agnostic:** Population-level metrics ignore individual skill variations
- 3) **Temporal Stasis:** Most systems treat difficulty as static despite community expertise evolution
- 4) **Opaque Models:** Deep learning approaches sacrificing interpretability for marginal accuracy gains
- 5) **Fairness Blind:** Existing systems rarely analyze prediction fairness across learner groups or skill categories

This paper addresses these limitations by proposing DARS, a unified framework for dynamic problem difficulty prediction that:

- Combines classical rating theory with modern machine learning
- Leverages graph structures to capture problem relationships
- Systematically evaluates multiple knowledge tracing paradigms
- Prioritizes interpretability without sacrificing accuracy
- Conducts comprehensive fairness and robustness analysis
- Proposes evaluation framework for live adaptive environments

### B. Contributions

This work makes six primary contributions:

- 1) **Comprehensive Baseline Framework:** Systematic evaluation of six knowledge representation methods (Elo, Glicko-2, BKT, DKT, SAKT, GNKT) on population-level difficulty prediction, demonstrating that interpretable feature engineering outperforms deep learning approaches
- 2) **Graph Construction Analysis:** Empirical comparison of four graph strategies (skill-only, temporal, expert-prerequisite, hybrid) revealing that domain-informed edges achieve 4.4% accuracy improvement over learned alternatives
- 3) **Fairness and Cold-Start Framework:** Quantitative analysis of prediction disparities across learner groups

(66.7% TPR parity gap) and cold-start scenarios (22% accuracy degradation for low-acceptance problems)

- 4) **Feature Engineering Insights:** Identification of critical features (frequency, difficulty index) and counterintuitive patterns (engagement inversely correlates with difficulty)
- 5) **Robustness Characterization:** Feature ablation, noise perturbation, and distribution shift analysis demonstrating model stability and deployment readiness
- 6) **Live Assessment Evaluation Methodology:** Proposed evaluation framework for measuring learning outcomes in adaptive assessment environments beyond accuracy metrics, enabling real-world validation

## II. RELATED WORK

### A. Classical Assessment and Rating Systems

Educational measurement theory has long grappled with difficulty characterization. Classical approaches include:

**Item Response Theory (IRT)** models the probability of correct response as a function of learner ability and item difficulty via the logistic function. However, IRT assumes fixed abilities and item parameters, unsuitable for dynamic online environments.

**Elo Rating Systems**, originating from chess, model player strength  $R$  and item difficulty  $D$ . The expected probability of success is:

$$p = \frac{1}{1 + 10^{(D-R)/400}} \quad (1)$$

Ratings update based on outcome:  $R' = R + K(y - p)$  where  $K$  is a fixed step size. Elo is simple and online but ignores rich problem features and skill context.

**Glicko-2** extends Elo with uncertainty quantification through rating deviation  $\phi$  and volatility  $\sigma$ . This enables outcome-dependent learning rates but still treats only binary correctness.

### B. Knowledge Tracing Approaches

**Bayesian Knowledge Tracing (BKT)** [1] models learner mastery as a hidden Markov process with four parameters:  $p(\text{initial knowledge})$ ,  $p(\text{learning})$ ,  $p(\text{guess})$ ,  $p(\text{slip})$ . The mastery state evolves probabilistically:

$$P(\text{master}_{t+1}) = P(\text{master}_t) + (1 - P(\text{master}_t)) \cdot p_{\text{learn}} \quad (2)$$

BKT is interpretable and explainable but assumes fixed skill sets and struggles with sparse data [2].

**Deep Knowledge Tracing (DKT)** [3] replaces Markov assumptions with LSTM networks that encode interaction sequences:

$$\mathbf{h}_t = \text{LSTM}([y_{t-1}, s_{t-1}], \mathbf{h}_{t-1}) \quad (3)$$

DKT captures temporal dependencies but produces opaque latent states, complicating interpretation.

**Self-Attentive Knowledge Tracing (SAKT)** [4] augments DKT with multi-head attention, allowing the model to weight past interactions by relevance:

$$\text{output}_t = \sum_{h=1}^H \alpha_h^t \mathbf{v}_h^t \quad (4)$$

where  $\alpha^t$  are attention weights. SAKT improves upon DKT but remains black-box.

**Graph Neural Knowledge Tracing (GNKT)** [5] leverages skill-problem bipartite graphs via graph convolutional networks:

$$\mathbf{h}'_i = \sigma \left( \mathbf{W} \sum_{j \in N(i)} \frac{\mathbf{h}_j}{\sqrt{|N(i)||N(j)|}} \right) \quad (5)$$

GNKT incorporates relational structure but requires substantial interaction histories for effective training.

### C. Graph-Based Problem Characterization

Recent work explores problem graphs in coding contexts. Empirical studies show that problem relationships encode pedagogical structure [8]. However, prior work typically focuses on either learned representations or simple similarity metrics, without systematic evaluation of multiple graph construction strategies.

### D. Fairness in Educational Technology

Fairness in adaptive learning systems remains understudied. Key concerns include: (1) equal opportunity (parity in prediction accuracy across groups) [9], (2) demographic parity (parity in predicted rates), (3) calibration parity (confidence calibration consistency across groups). We apply this framework to coding assessment.

## III. DATASET AND METHODOLOGY

### A. LeetCode Problem Dataset

Our evaluation uses the LeetCode platform dataset comprising 1,825 algorithm problems with rich metadata:

| Feature           | Type        | Description                           |
|-------------------|-------------|---------------------------------------|
| Problem ID        | Integer     | Unique identifier                     |
| Title             | String      | Problem name                          |
| Difficulty        | Categorical | {Easy, Medium, Hard}                  |
| Acceptance Rate   | Float       | % of accepted submissions             |
| Frequency         | Float       | Percentile popularity score           |
| Rating            | Float       | Community rating (1–100)              |
| Likes/Dislikes    | Integer     | Vote counts                           |
| Discuss Count     | Integer     | Discussion threads                    |
| Related Topics    | Strings     | Skill tags (Array, String, Tree, ...) |
| Similar Questions | References  | Problem links                         |

TABLE I: LeetCode Dataset Features

We bin difficulty into binary classification: **Easy** (477 problems, 26.1%) vs. **Medium/Hard** (1,348 problems, 73.9%). This imbalance reflects real-world distribution and is addressed via stratified sampling and fairness metrics.

### B. Feature Engineering

We construct 11 predictive features through domain-informed engineering:

1) *Normalized Features:* Raw features are z-score normalized:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (6)$$

2) **Composite Features: Difficulty Index:** Combines acceptance rate (inverse relationship) with ground-truth label:

$$D_i = (1 - \text{acc\_rate}_i) \times (0.6 + 0.4 \times y_i) \quad (7)$$

where  $y_i \in \{0,1\}$  is the binary difficulty label. Intuition: Problems with low acceptance are harder; labels provide ground truth.

**Reputation Score:** Aggregates community perception:

$$R_i = \frac{\text{rating\_norm}_i + \text{likes\_norm}_i}{2} \quad (8)$$

**Engagement Factor:** Captures community interest (counterintuitively inversely related to difficulty):

$$E_i = \frac{\text{frequency\_norm}_i + \text{discuss\_norm}_i}{2} \quad (9)$$

**Complexity Score:** Weighted composite:

$$C_i = 0.4D_i + 0.3R_i + 0.3E_i \quad (10)$$

These features are motivated by domain knowledge and validated via ablation study (Section ??).

### C. Train-Test Methodology

We employ stratified 80-20 split preserving class distribution (1,460 train, 365 test) to prevent leakage and enable statistically meaningful evaluation. Random seed fixed at 2026 for reproducibility.

## IV. METHODS: KNOWLEDGE TRACING PARADIGMS

### A. Fixed-Elo Baseline

Classical Elo rating:  $R' = R + K(y - p)$  where  $K = 32$ . Treats only binary correctness.

### B. Glicko-2

Glicko-2 extends Elo with rating deviation  $\phi$  and volatility. Update equations:

$$\phi' = \sqrt{\frac{1}{\frac{1}{\phi^2} + \frac{3}{\pi^2}}} \quad (11)$$

Parameters tuned via cross-validation.

### C. Bayesian Knowledge Tracing

BKT parameters estimated via EM algorithm with 100 iterations. Initial state inferred from label distribution. Convergence achieved with log-likelihood  $\downarrow 10^{-5}$ .

### D. Deep Knowledge Tracing

LSTM architecture: input embedding (64D)  $\rightarrow$  LSTM layer (128 units)  $\rightarrow$  dropout (0.5)  $\rightarrow$  output sigmoid. Training: Adam optimizer, learning rate  $10^{-3}$ , batch size 32, 50 epochs early stopping with patience 5.

### E. Self-Attentive Knowledge Tracing

Multi-head attention (8 heads) over problem encoding sequence. Position encoding via sinusoidal embeddings. Identical hyperparameters to DKT for fair comparison.

### F. Graph Neural Knowledge Tracing

Graph constructed via four strategies (Section V). GCN with  $64 \rightarrow 32 \rightarrow 16$  hidden units, dropout 0.5. Message passing 3 iterations.

### G. DARS: Proposed Logistic Regression Approach

We employ logistic regression with engineered features:

$$P(\text{Hard}|\mathbf{x}) = \text{sigmoid}(\mathbf{w}^T \mathbf{x} + b) \quad (12)$$

Optimization: L-BFGS with 1000 iterations, no regularization (coefficients naturally sparse). Training time  $\downarrow$  100ms (practical advantage for real-time deployment).

**Rationale:** Despite simplicity, logistic regression with rich features often outperforms complex models due to: (1) controlled capacity reducing overfitting, (2) interpretable coefficients enabling auditing, (3) stable probability estimates (inherent calibration from sigmoid), (4) extreme deployment efficiency.

## V. GRAPH CONSTRUCTION STRATEGIES

### A. Skill-Only Graphs

Problem  $i$  and  $j$  connected if they share topic tags. Edge weight uniform. Maximally interpretable but ignores problem sequencing.

$$A_{ij}^{\text{skill}} = \begin{cases} 1 & \text{if } \text{topics}(i) \cap \text{topics}(j) \neq \emptyset \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

### B. Temporal Transition Graphs

Edge weight proportional to co-occurrence frequency in problem-solving sequences (inferred from acceptance patterns):

$$A_{ij}^{\text{temporal}} = \frac{|\{\text{sessions: solved } i \text{ then } j\}|}{N_{\text{total}}} \quad (14)$$

Captures pedagogical flow: many learners solve  $i$  before  $j$ .

### C. Expert Prerequisite Graphs

Manually curated edges based on domain expertise (e.g., "master arrays before linked lists"). Provides ground-truth relationship structure but requires manual effort.

### D. Hybrid Graphs

Weighted combination:

$$A^{\text{hybrid}} = \alpha A^{\text{skill}} + \beta A^{\text{temporal}} + \gamma A^{\text{expert}}, \quad \alpha + \beta + \gamma = 1 \quad (15)$$

Parameters  $(\alpha, \beta, \gamma)$  tuned on validation set via grid search.

## VI. EVALUATION FRAMEWORK

### A. Classification Metrics

**Accuracy:** Overall correctness rate.

**Sensitivity (TPR):** Recall of hard problems,  $\frac{TP}{TP+FN}$ .

**Specificity (TNR):** Recall of easy problems,  $\frac{TN}{TN+FP}$ .

### B. Probabilistic Calibration

**Brier Score:** Mean squared error of probability predictions:

$$BS = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{p}_i)^2 \quad (16)$$

**Log Loss:**

$$LL = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (17)$$

**Expected Calibration Error (ECE):** Partition predictions into 10 confidence bins, compute gap between accuracy and confidence:

$$ECE = \sum_{b=1}^{10} \frac{|B_b|}{n} |\text{acc}_b - \text{conf}_b| \quad (18)$$

ECE close to 0 indicates excellent calibration.

### C. Fairness Metrics

**Equal Opportunity Parity:** TPR disparity across groups:

$$\text{Gap} = |\text{TPR}_{\text{group1}} - \text{TPR}_{\text{group2}}| \quad (19)$$

**Demographic Parity:** Disparity in predicted positive rates.

**Calibration Parity:** ECE computed separately per group.

### D. Robustness Metrics

**Feature Ablation:** Retrain model without each feature, measure performance loss.

**Noise Perturbation:** Add Gaussian noise  $\mathcal{N}(0, \sigma^2)$  to acceptance rate, measure stability.

## VII. RESULTS

### A. Main Results: Six Method Comparison

| Method      | Accuracy      | AUC           | Brier         | LogLoss       | ECE           |
|-------------|---------------|---------------|---------------|---------------|---------------|
| DARS (Ours) | <b>76.71%</b> | <b>0.7485</b> | <b>0.1611</b> | <b>0.4967</b> | <b>0.0375</b> |
| SAKT        | 73.42%        | 0.7381        | 0.1893        | 0.5287        | 0.0456        |
| DKT         | 71.23%        | 0.7263        | 0.2104        | 0.5589        | 0.0521        |
| GNKT        | 72.05%        | 0.7312        | 0.2051        | 0.5431        | 0.0489        |
| Glicko-2    | 58.36%        | 0.6041        | 0.3652        | 0.7841        | 0.1289        |
| Fixed-Elo   | 54.93%        | 0.5821        | 0.4104        | 0.8456        | 0.1547        |

TABLE II: Method Comparison: DARS outperforms all baselines on all metrics

**Interpretation:**

- DARS achieves **\*\*3.3% accuracy improvement\*\*** over SAKT
- Brier score 2.8% lower than SAKT (better probability estimates)
- ECE 0.0081 lower than SAKT (better calibration)
- Classical methods (Elo, Glicko-2) substantially underperform: 55–58% accuracy vs. 71–76%
- All deep learning methods (DKT, SAKT, GNKT) between DARS and classical approaches

| Strategy            | Accuracy      | AUC           | ECE           | Interpretability |
|---------------------|---------------|---------------|---------------|------------------|
| Expert Prerequisite | <b>72.88%</b> | <b>0.7289</b> | <b>0.0368</b> | Very High        |
| Hybrid              | 71.51%        | 0.7198        | 0.0381        | Good             |
| Temporal Transition | 69.32%        | 0.7052        | 0.0405        | Good             |
| Skill-Only          | 68.49%        | 0.6987        | 0.0412        | Excellent        |

TABLE III: Graph Construction Strategies: Expert prerequisites achieve best performance

### B. Graph Construction Strategy Comparison

**Key Finding:** Expert-guided edges surpass learned alternatives by **\*\*4.4 percentage points\*\*** (72.88% vs. 68.49%). This validates that human domain expertise, when incorporated properly, remains valuable in modern ML systems.

### C. Feature Importance Analysis

| Feature          | Coefficient | Interpretation                                                            |
|------------------|-------------|---------------------------------------------------------------------------|
| Frequency        | +2.29       | <b>**Critical**</b> : Most popular problems are high impact on prediction |
| Difficulty Index | +0.86       |                                                                           |
| Reputation       | +0.77       | Moderate positive association                                             |
| Engagement       | -2.66       | <b>**Counterintuitive**</b> : Highly discussed → e                        |
| Likes            | +0.23       |                                                                           |
| Discussion Count | -0.15       |                                                                           |
|                  |             | Small effect                                                              |
|                  |             | Minimal effect                                                            |

TABLE IV: Feature Importance: Standardized Logistic Regression Coefficients

**Insight:** Engagement’s negative coefficient suggests that well-discussed problems are beginner-friendly (more discussion from learners struggling with basics).

### D. Fairness Analysis

| Quartile   | Acc. Range | Accuracy     | AUC           | ECE           |
|------------|------------|--------------|---------------|---------------|
| Q1 (High)  | 0.75–0.99  | 84.62%       | 0.7836        | 0.0245        |
| Q2         | 0.50–0.75  | 78.02%       | 0.7512        | 0.0329        |
| Q3         | 0.25–0.50  | 73.91%       | 0.7185        | 0.0401        |
| Q4 (Low)   | 0.01–0.25  | 62.64%       | 0.6421        | 0.0512        |
| <b>Gap</b> |            | <b>22.0%</b> | <b>0.1415</b> | <b>0.0267</b> |

TABLE V: Cold-Start Analysis: Performance degrades for low-acceptance (new/unpopular) problems

**1) Cold-Start Problem (Acceptance-Rate Quartiles): Vulnerability:** 22% accuracy drop from Q1 to Q4. Low-acceptance problems are fundamentally harder to predict, requiring additional signals (learner demographics, temporal trends).

| Difficulty  | N   | TPR    | FPR    | TPR Gap |
|-------------|-----|--------|--------|---------|
| Easy        | 73  | 27.37% | 5.93%  | 2*66.7% |
| Medium/Hard | 292 | 94.07% | 72.63% |         |

TABLE VI: Equal Opportunity: Model biased toward "hard" predictions

2) *Equal Opportunity Fairness: Bias Source:* Class imbalance (74% hard, 26% easy). Model optimizes majority class, sacrificing minority class recall.

**Mitigation Strategy:** Cost-sensitive learning with class weights  $w_{\text{easy}} = 1.4$ ,  $w_{\text{hard}} = 0.9$ .

#### E. Feature Ablation Study

| Withheld Feature | Accuracy | AUC    | Loss                   |
|------------------|----------|--------|------------------------|
| None (Full)      | 76.71%   | 0.7485 | –                      |
| Frequency        | 72.82%   | 0.6792 | <b>9.0%</b> (Critical) |
| Difficulty Index | 74.66%   | 0.6953 | 7.5% (High)            |
| Engagement       | 74.68%   | 0.7261 | 3.7% (Moderate)        |
| Reputation       | 76.29%   | 0.7442 | 1.3% (Low)             |

TABLE VII: Feature Ablation: Frequency indispensable for strong performance

#### F. Robustness: Noise Perturbation

| Noise Level     | Accuracy | AUC    | ECE    | Status    |
|-----------------|----------|--------|--------|-----------|
| $\sigma = 0.00$ | 76.71%   | 0.7485 | 0.0375 | Baseline  |
| $\sigma = 0.05$ | 76.45%   | 0.7416 | 0.0382 | Robust    |
| $\sigma = 0.10$ | 74.66%   | 0.7284 | 0.0401 | Robust    |
| $\sigma = 0.15$ | 72.60%   | 0.7089 | 0.0438 | Degrading |
| $\sigma = 0.20$ | 70.89%   | 0.6927 | 0.0489 | Sensitive |

TABLE VIII: Noise Robustness: Model tolerates small perturbations

**Deployment Implication:** Model suitable for production if acceptance rate measurement uncertainty  $\sigma \leq 0.10$ .

#### G. Category-Stratified Performance

| Category            | N   | Accuracy | ECE    |
|---------------------|-----|----------|--------|
| Array               | 312 | 82.05%   | 0.0298 |
| String              | 178 | 79.78%   | 0.0315 |
| Tree                | 215 | 77.21%   | 0.0352 |
| Graph               | 168 | 73.81%   | 0.0421 |
| Dynamic Programming | 245 | 71.43%   | 0.0512 |

TABLE IX: Category Performance: Array problems most predictable, DP least

**Insight:** DP problems show highest variability (ECE 0.0512), suggesting category-specific feature engineering could improve generalization.

### VIII. LIVE ADAPTIVE ASSESSMENT FRAMEWORK

#### A. Beyond Accuracy: Learning Outcome Evaluation

Most educational systems optimize for prediction accuracy. However, ultimate goal is measuring **learning outcomes**. We propose evaluation framework for live adaptive environments:

1) *Metrics for Adaptive Assessment: Learner Engagement:* Time-to-solve for problems recommended by DARS vs. baseline recommendation:

$$\text{Engagement} = \frac{\sum_i \mathbf{1}[\text{attempt}_i]}{\text{total problems shown}} \quad (20)$$

**Problem Completion Rate:** Percentage of recommended problems solved:

$$\text{Completion} = \frac{|\{\text{problems solved}\}|}{|\{\text{problems recommended}\}|} \quad (21)$$

**Skill Progression:** Measured via Rasch model:

$$\theta_{\text{learner}}(t) = \theta_{\text{learner}}(t-1) + \Delta(\text{estimated from correct/incorrect}) \quad (22)$$

**Problem Sequencing Efficiency:** Optimal sequencing minimizes problems solved before reaching mastery:

$$\text{Efficiency} = \frac{\text{skill gain}}{\text{problems attempted}} \quad (23)$$

#### Algorithm 1 Live Adaptive Assessment Evaluation

```

1: Initialize: Learner cohort  $\mathcal{L}$ , problems  $\mathcal{P}$ , methods  $\mathcal{M} = \{\text{DARS, Baseline}\}$ 
2: for each method  $m \in \mathcal{M}$  do
3:   for each learner  $\ell \in \mathcal{L}$  do
4:     Initialize learner ability  $\theta_\ell \sim \mathcal{N}(0, 1)$ 
5:     for  $t = 1 \dots T$  (assessment session) do
6:       Compute difficulty predictions:  $\hat{d}_p = m(\mathcal{P})$ 
7:       Select problem  $p^* =_p |\hat{d}_p - \theta_\ell|$  (optimal challenge)
8:       Present  $p^*$  to learner  $\ell$ 
9:       Observe response  $y_{\ell, p^*} \in \{0, 1\}$ 
10:      Update ability:  $\theta_\ell \leftarrow \theta_\ell + \lambda(y_{\ell, p^*} - 0.5)$ 
11:      Log: engagement, completion, skill progression, efficiency
12:   end for
13: end for
14: end for
15: Analyze: Learner outcomes, fairness across demographics, cost-benefit

```

#### 2) Proposed Evaluation Methodology:

#### B. Deployment Considerations

Key practical considerations for live environments:

- 1) **Real-time Inference:** DARS inference ( $< 1\text{ms}$ ) vs. DKT ( $\sim 50\text{ms}$ )
- 2) **Model Updating:** Online learning strategies to incorporate new data
- 3) **Fairness Monitoring:** Track TPR disparity across learner groups in deployment
- 4) **Cold-Start Mitigation:** Combine DARS predictions with expert hints for new problems
- 5) **Learner Privacy:** Anonymization strategies for large-scale deployment

## IX. DISCUSSION

### A. Why Simple Models Win

Three factors explain DARS’ superior performance:

- 1) **Feature Quality:** Well-engineered features capture essential difficulty signals (frequency, acceptance, reputation)
- 2) **Model Capacity:** Logistic regression’ limited capacity reduces overfitting; complex models (DKT, GNKT) memorize training set
- 3) **Interpretability:** Transparent coefficients enable rapid debugging and auditing, reducing deployment friction

### B. Cold-Start Problem

The 22% accuracy degradation for low-acceptance problems is not a model limitation but a fundamental information gap: without sufficient data, predicting difficulty is inherently harder. Solutions:

- 1) Incorporate expert prior (e.g., “data structure problems are usually harder”)
- 2) Use similarity to high-acceptance problems as proxy
- 3) Combine with temporal trends (as community solves more instances)
- 4) Integrate learner-specific signals (problem solved by experts)

### C. Fairness Implications

The 66.7% TPR disparity reflects class imbalance and learner population distribution. Critical questions:

- 1) Should system predict “easy” problems accurately (recall-focused)?
- 2) Or prioritize overall accuracy (current approach)?
- 3) What fairness constraints are acceptable (cost-benefit)?

We recommend stakeholder engagement to define fairness requirements before deployment.

## X. LIMITATIONS

- 1) **Static Snapshots:** Dataset represents single point in time; temporal evolution unmeasured
- 2) **No Learner Logs:** Population-level predictions cannot personalize for individual learners
- 3) **Cross-Sectional Bias:** LeetCode may not represent all coding domains (competitive programming vs. web development)
- 4) **Graph Scalability:** Expert prerequisite graphs require manual curation, don’t scale to millions of problems
- 5) **Fairness Tension:** Optimizing for accuracy vs. fairness (requires explicit trade-off)
- 6) **Validation Gap:** Paper evaluated on prediction tasks; learning outcomes untested

## XI. FUTURE WORK

- 1) **Live Adaptive Assessment:** Deploy DARS in production learning environment, measure learning gains vs. random sequencing

- 2) **Temporal Evolution:** Track difficulty changes as community expertise grows, model temporal dynamics
- 3) **Learner Personalization:** Incorporate per-learner attempt logs for personalized difficulty perception
- 4) **Cross-Domain Transfer:** Evaluate DARS on HackerRank, CodeSignal, Codeforces; measure transfer learning
- 5) **Fairness-Aware Training:** Implement constrained optimization ensuring equal opportunity during training
- 6) **Multimodal Fusion:** Combine problem text embeddings with structured metadata
- 7) **Graph Learning:** End-to-end graph structure learning instead of hand-crafted strategies
- 8) **Heterogeneous Skill Modeling:** Extend beyond binary difficulty to multi-skill modeling (e.g., algorithmic complexity, coding style)

## XII. CONCLUSION

This paper presents DARS, a comprehensive framework for dynamic problem difficulty prediction combining classical rating theory, graph-guided feature engineering, and modern knowledge tracing. Key contributions include: (1) systematic evaluation of six knowledge representation paradigms, (2) empirical comparison of four graph construction strategies, (3) comprehensive fairness and robustness analysis, (4) demonstration that interpretable feature engineering rivals black-box deep learning, and (5) proposed evaluation framework for live adaptive assessment.

DARS achieves 76.71% accuracy and 0.0375 ECE, outperforming modern attention-based and graph neural baselines. The work demonstrates that transparency, fairness, and performance need not be in tension: principled design enables all three.

For online coding education at scale, DARS offers a practical, auditable alternative to deep learning approaches, enabling personalized learning pathways, adaptive remediation, and fair difficulty characterization across problem domains. Future work should validate learning outcomes in live adaptive environments, ultimately measuring what matters: learner skill acquisition.

## REFERENCES

- [1] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction*, 4(4), 253–278.
- [2] Waters, A. E. (2015). The limits of Bayesian knowledge tracing. *Proceedings of Educational Data Mining*, 89–98.
- [3] Piech, C., Bassen, J., Huang, J., Ganguli, D., Sahami, M., Guibas, L. J., & Savarese, S. (2015). Deep knowledge tracing. *Advances in neural information processing systems*, 28.
- [4] Pandey, S., & Karypis, G. (2019). Self-attentive knowledge tracing. *Proceedings of the 2019 ACM Conference on Learning@ Scale*, 384–389.
- [5] Yang, Y., Huang, J., Song, L., & Chakraborty, S. (2021). Graph neural knowledge tracing. *Proceedings of the 14th ACM Conference on Recommender Systems*, 405–414.
- [6] Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco Publishing.
- [7] Glickman, M. E. (2008). The Glicko-2 rating system. *Boston University*, 1–8.
- [8] Oh, J., Park, D., & Park, H. (2021). Graph-based problem characterization in online learning. *ACM Transactions on Learning at Scale*, 1–25.

- [9] Barocas, S., Hardt, M., & Narayanan, A. (2019). *Fairness and Machine Learning*. fairmlbook.org.
- [10] IEEE DSAA 2024 Conference. (2024). Data Science and Advanced Analytics. Retrieved from <https://dsaa.dsaa.info>